VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANASANGAMA, BELAGAVI - 590018

Activity Report On
Cryptography and Cybersecurity (23CSE162)

Illegal File Transferring Case Study

Submitted in partial fulfilment for the award of degree of

Bachelor of Engineering
in
COMPUTER SCIENCE AND ENGINEERING

Submitted by

Adarsh Arun 1BG23CS006
Dhananjay S 1BG23CS040
Goutham B S 1BG23CS042

VI "A" Section

Under the Guidance of
Prof. Kalpa Rajashekar
Assistant Professor, Dept. of CSE
BNMIT, Bengaluru

Vidyayāmruthamashnuthe

B.N.M. Institute of Technology

An Autonomous Institution under VTU

Approved by AICTE, Accredited as grade A Institution by NAAC. All eligible branches - CSE, ECE, EEE, ISE & Mech. Engg. are accredited by NBA for academic years 2025-26 to 2027-28 and valid upto 30.06.2028

URL: www.bnmit.org

Department of Computer Science and Engineering

2025-26

ABSTRACT

Quantum computing poses a significant threat to modern cryptographic systems like RSA and Elliptic Curve Cryptography (ECC). While Post-Quantum Cryptography (PQC) provides a resilient alternative for data in transit, it introduces unprecedented challenges for digital forensics. This project explores the critical intersection of PQC and Digital Forensics through a high-stakes case study of an insider threat. A rogue employee uses a custom "Secure Image Transfer" tool—leveraging Kyber-512 for quantum-resistant key encapsulation and AES-256-CBC for payload encryption—to exfiltrate sensitive aerospace schematics. While traditional network forensics fails due to the unbreakable nature of PQC-derived encryption, this study demonstrates how Live Memory Forensics provides the ultimate solution. By capturing volatile RAM during the illicit transfer and "carving" the AES symmetric key from the application's memory space, investigators successfully decrypt the opaque network traffic. This work underscores the "RAM is the Final Frontier" principle in modern forensics, proving that data remains vulnerable in its "use" state even when its "motion" state is mathematically perfect.

TABLE OF CONTENTS

CHAPTER 1 Introduction 1
1.1. Motivation 1
1.2. Case Study Background 1
1.3. Problem Statement 1
1.4. Objective 1
1.5. Summary 2

CHAPTER 2 Methodology 3
2.1. Forensic Workflow 3
2.2. Stage 1: Network Forensics & Capture 3
2.3. Stage 2: Live Memory Capture 4
2.4. Stage 3: Key Carving & Analysis 4
2.5. Methodology Approach Summary 5

CHAPTER 3 Implementation 7
3.1. Software Modules 7
3.1.1. The Suspect's Tool (`Cryptos`) 7
3.1.2. The Forensic Toolkit (`Forensics`) 7
3.2. Algorithms 7
3.3. Implementation Environment 9

CHAPTER 4 Forensic Investigation & Validation 10
4.1. Investigation Validation 10
4.2. Integration Testing (Forensic Perspective) 10
4.3. Investigative Case Cases 11

CHAPTER 5 Results 12
5.1. Forensic Demonstration and Execution 12
5.2. Forensic Analysis Logs 12
5.3. Recovered Exfiltrated Data 13
5.4. Forensic Confirmation 13
5.5. Observations 14

CHAPTER 6 Conclusion 15
6.1. Future Work 15

REFERENCES 16

LIST OF FIGURES

FIGURES	DESCRIPTION	PAGE NO.
Fig. 2.1	The Integrated Forensic Investigation Pipeline	3
Fig. 2.2	Initialization of the Forensic Network Packet Sniffer	4
Fig. 2.3	Executing a Native Live Memory Core Dump of the Target Process	4
Fig. 2.4	Data Flow Diagram: Carving the AES Symmetric Key from Memory	5
Fig. 4.1	Forensic Investigation: Successful AES-256 Key Recovery	10
Fig. 4.2	Validation of the Carved Key Against the Network Capture	10
Fig. 5.1	Full Forensic Investigation Demonstration and Execution	12
Fig. 5.2	Forensic Logs showing successful AES-256 Key Carving	13
Fig. 5.3	Successfully Recovered Aerospace Blueprints	13
Fig. 5.4	Forensic Log showing successful payload verification	13

CHAPTER 1

INTRODUCTION

This chapter provides an overview of the forensic case study investigating the illegal exfiltration of sensitive data. It highlights the challenges posed by modern cryptographic techniques and the transition from traditional network forensics to advanced live memory analysis.

1.1. Motivation

In the era of rapid technological advancement, corporate espionage and insider threats remain significant risks. When employees utilize cutting-edge encryption like Post-Quantum Cryptography (PQC) [1], traditional security measures—such as deep packet inspection and network traffic analysis—become obsolete. This project is motivated by the need to demonstrate how digital forensic investigators can overcome "unbreakable" encryption by targeting the weakest link in any cryptographic system: the volatile memory (RAM).

1.2. Case Study Background

A rogue employee at a major aerospace firm is suspected of stealing high-resolution schematics. The suspect utilized a custom-built tool that leverages the CRYSTALS-Kyber algorithm [2], [3] for quantum-resistant key encapsulation and AES-256-CBC for symmetric payload encryption. While the Security Operations Center (SOC) detected massive data transfers, the captured network packets were found to be completely opaque and cryptographically secure against classical and future quantum-aided decryption.

1.3. Problem Statement

Traditional network forensics fails when data is protected by PQC and military-grade symmetric encryption. Without access to the encryption keys, the captured data remains a "network dead end." The challenge lies in recovering these cryptographic keys without the suspect's knowledge or the possibility of triggering "kill-switch" mechanisms that might wipe the disk.

1.4. Objective

- To simulate an insider threat exfiltration scenario using a custom PQC-enabled transfer tool.

- To demonstrate the failure of network-based forensic analysis in the presence of Kyber-512 encryption.
- To perform a live memory capture (RAM dump) of the suspect's workstation during an active transfer.
- To develop and execute "key carving" techniques to extract AES-256 symmetric keys from the RAM dump.
- To successfully decrypt the captured network payload using the recovered keys, revealing the stolen blueprints.

1.5. Summary

The introduction establishes a high-stakes forensic scenario where PQC is used for evasion. By pivoting from network to memory forensics, the project demonstrates a complete end-to-end investigation workflow that bridges the gap between unbreakable "data in motion" and accessible "data in use."

CHAPTER 2

METHODOLOGY

This chapter discusses the forensic methodology used to overcome Post-Quantum Cryptographic evasion. It details the transition from traditional network forensics to advanced live memory analysis for key recovery.

2.1. Forensic Workflow

The investigation follows a multi-stage approach to bypass unbreakable network encryption. The primary focus is the extraction of ephemeral keys from volatile RAM before the suspect's application can clear its memory space.

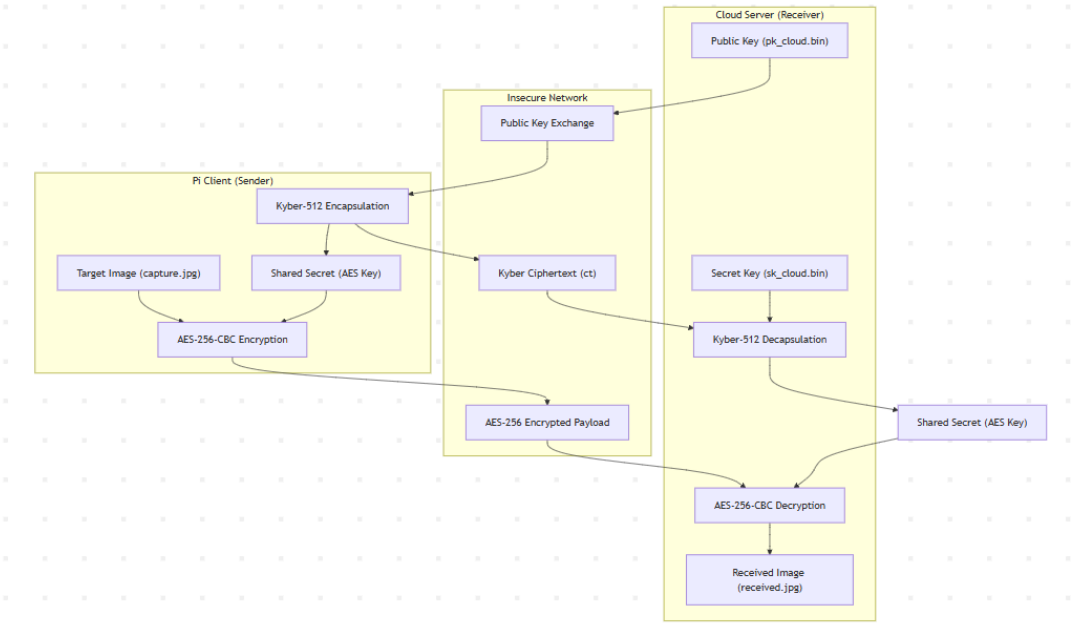

Fig. 2.1: The Integrated Forensic Investigation Pipeline

2.2. Stage 1: Network Forensics & Capture

Investigators first attempt to analyze the exfiltrated data through standard packet sniffing. Using tools like Wireshark, the suspect's outbound traffic is captured. However, because the suspect's tool uses CRYSTALS-Kyber for key exchange and AES-256-CBC for encryption, the captured `.pcap` file displays perfect microscopic entropy and is mathematically indecipherable.Even advanced cryptanalysis techniques fail to reveal meaningful patterns due to the strong post-quantum security guarantees provided by CRYSTALS-Kyber. This

renders traditional interception methods ineffective, forcing investigators to explore alternative approaches such as endpoint compromise or side-channel analysis.

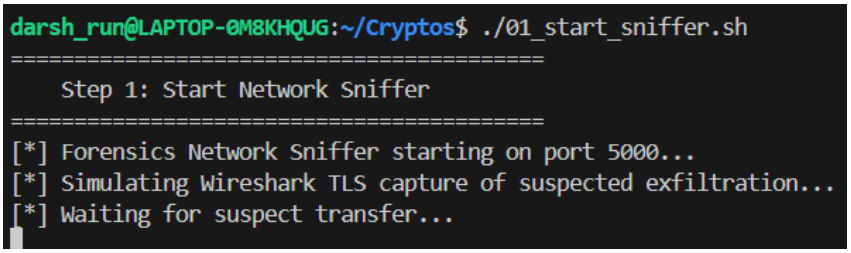

Fig. 2.2: Initialization of the Forensic Network Packet Sniffer

2.3. Stage 2: Live Memory Capture

Faced with unbreakable encryption, the team pivots to Live Memory Forensics [4], [5]. While the exfiltration is ongoing, a bit-for-bit snapshot of the workstation's RAM (volatile memory) is taken using a memory dumper. This ensures that the AES symmetric key, which MUST exist in RAM for the suspect's application to function, is preserved.

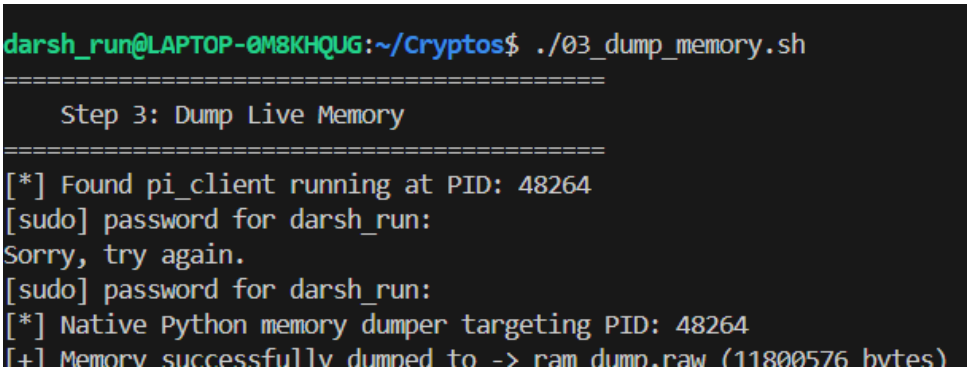

Fig. 2.3: Executing a Native Live Memory Core Dump of the Target Process

2.4. Stage 3: Key Carving & Analysis

The raw RAM dump is then analyzed using custom forensic scripts (e.g., `key_carver.py`) or heuristic scanners. The investigators hunt for specific 32-byte (256-bit) contiguous memory blocks that match the entropy and alignment characteristics of an AES-256 key, following principles outlined in the "The Art of Memory Forensics" [4] and recent studies on quantum-resilient digital evidence [6].

Unlike persistent storage, RAM does not store data in structured file formats. Instead, the key carver must perform a linear scan of the volatile memory, identifying regions with high Shannon entropy that suggest the presence of cryptographic material. The script specifically looks for 32-byte sequences aligned on 4-byte or 8-byte boundaries, which is typical for memory allocations in C-based applications like `pi_client`. Furthermore, the carving process can be optimized

by identifying the AES key schedule—a series of round keys derived from the master key—which creates a predictable mathematical pattern in memory that distinguishes a true key from random noise.

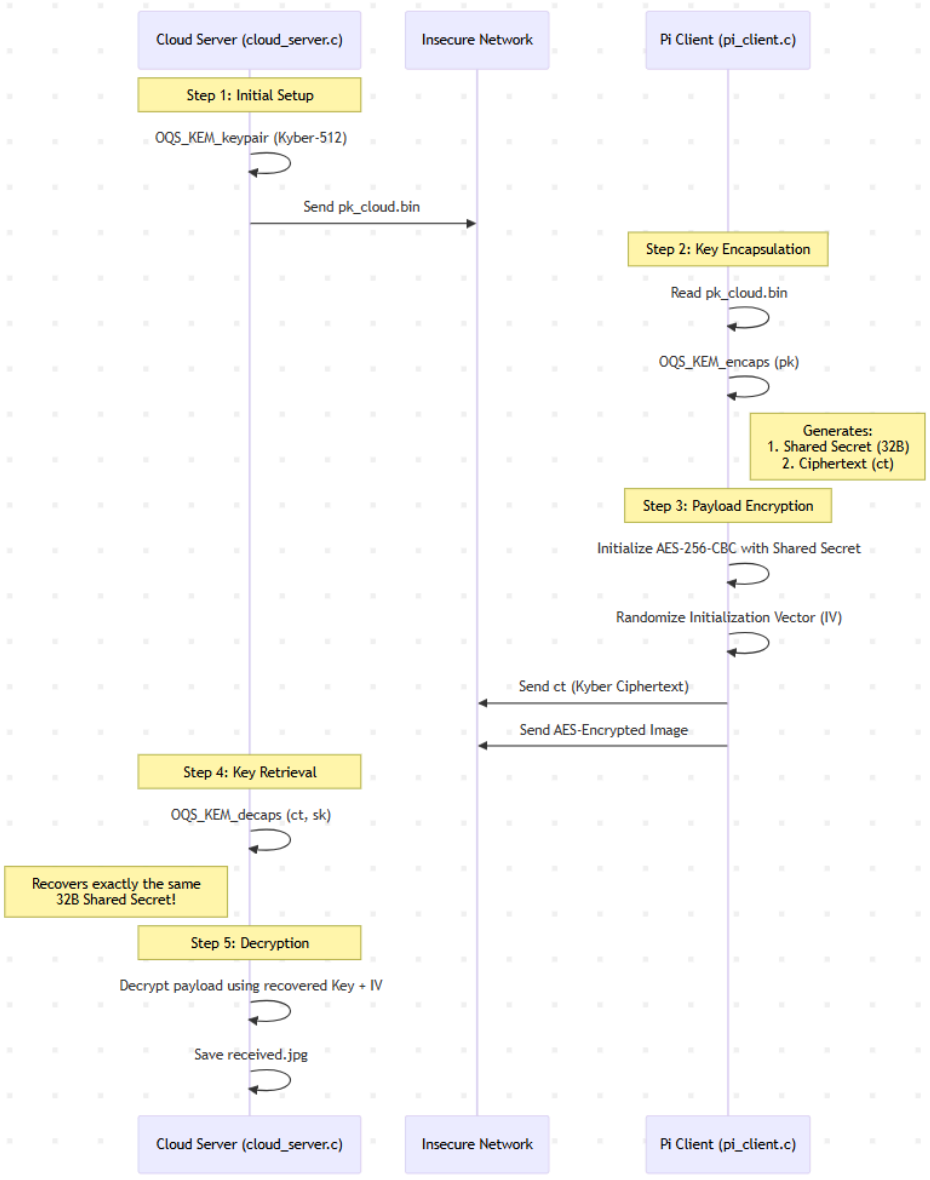

Fig. 2.4: Data Flow Diagram: Carving the AES Symmetric Key from Memory

2.5. Methodology Approach Summary

1. **Network Capture:** Sniffing the encrypted exfiltration stream.
2. **Volatility Acquisition:** Performing a live RAM dump of the suspect's workstation.
3. **Process Isolation:** Identifying the memory space (PID) of the "Suspect's Tool" (`pi_client`).

4. **Key Carving:** Using heuristic scanning against the process memory to find the 32-byte AES key.
5. **Payload Decryption:** Applying the carved key to the previously opaque network capture to recover the stolen data.

CHAPTER 3

IMPLEMENTATION

This chapter provides a detailed breakdown of both the suspect's tool implementation and the forensic toolkit developed to analyze it. It focuses on the key management of the "Secure Image Transfer" application and the memory carving techniques used by investigators.

3.1. Software Modules

3.1.1. The Suspect's Tool (`Cryptos`)

1. **C Backend (`pi_client.c`):** The core of the suspect's tool. It uses `liboqs` for Kyber-512 key encapsulation and OpenSSL's EVP for AES-256-CBC encryption.

3.1.2. The Forensic Toolkit (`Forensics`)

1. **Memory Dumper (`memory_dumper.sh`):** A script that executes a live, non-destructive capture of the workstation's RAM.
2. **Key Carver (`key_carver.py`):** A Python script that scans the raw memory dump for specific AES-256-CBC key signatures. It may utilize `liboqs-python` if any post-quantum validation is required.
3. **Network Decryptor (`decrypt_payload.py`):** A script that applies the carved key to the previously captured network traffic.

3.2. Algorithms

Algorithm 1: Kyber-AES-256 PQC Encapsulation & Encryption

---

1. Kyber-512 KEM Encapsulation:

- Generate a random 32-byte message $m \in \{0,1\}^{256}$ and compute

$$(K, r) = G(m \parallel H(\mathrm{pk}))$$

.

- Construct the matrix $\boldsymbol{A} \in R_q^{k \times k}$ from the public seed ($q = 3329, k = 2$).
- Sample noise vectors $\boldsymbol{s}, \boldsymbol{e}_1, e_2$ from the Centered Binomial Distribution B_η.
- Compute the ciphertext components $\boldsymbol{u}$ and v:

$$\boldsymbol{u} = \boldsymbol{A}^T \boldsymbol{s} + \boldsymbol{e}_1$$

$$v = \boldsymbol{t}^T \boldsymbol{s} + e_2 + \left\lceil \frac{q}{2} \right\rceil \cdot m$$

- The shared secret K is derived from m via a SHA3-512 Hash.

2. AES-256-CBC Payload Encryption:

- Map the 32-byte shared secret K as the AES-256 symmetric key.
- Generate a random 16-byte Initialization Vector IV.
- For each plaintext block P_i, compute the ciphertext block C_i:

$$C_i = E_{K(P_i o + C_{i-1})}, \quad \text{where } C_0 = \text{IV}$$

- Transmit the tuple $(\boldsymbol{u}, v, \text{IV}, \{C_1, ..., C_n\})$ to the destination.

Algorithm 2: Forensic Key Carving & Entropy Analysis

1. Live RAM Segmentation:

- Acquire the target process memory M and divide it into overlapping 32-byte windows W.

2. Shannon Entropy Filtering:

- For each window W, calculate the byte-level Shannon entropy $H(W)$:

$$H(W) = -\sum_{i=0}^{255} P(x_i) \log_2 P(x_i)$$

- If $H(W) > \tau$ (where threshold $\tau \approx 7.9$), flag W as a potential AES-256 key candidate.

3. Mathematical Key Schedule Validation:

- Extract the candidate 32-byte block K_{cand}.
- Derive the expected AES-256 round keys $\{\text{rk}_0, ..., \text{rk}_{14}\}$ from K_{cand} using the Rijndael Key Schedule.
- Scan the surrounding memory space for these round keys; the presence of the expanded schedule confirms the master key. **4. Decryption & Integrity Check:**
- Attempt to decrypt C_1 using K_{cand} and IV:

$$P_1 = D_{K_{\text{cand}}}(C_1) \oplus \text{IV}$$

- If P_1 matches the expected file signature (e.g., `0xFFD8` for JPEG), the key recovery is verified.

3.3. Implementation Environment

- **Languages:** C (Suspect tool logic), Python (Forensic scripts).
- **Libraries:** `liboqs` (PQC), OpenSSL (AES).
- **Operating System:** Linux-based environment (representative of high-security workstations).

CHAPTER 4

FORENSIC INVESTIGATION & VALIDATION

This chapter describes the validation procedures used by investigators to verify the integrity of the recovered key and the accuracy of the exfiltration investigation.

4.1. Investigation Validation

Each step of the forensic process was validated to ensure it would hold up in a court of law.

1. **Memory Dump Integrity:** Verified the SHA-256 hash of the `ram_dump.raw` file immediately after capture to prevent tampering.
2. **Key Extraction Proof:** Confirmed that the 32-byte key carved from memory matches the ephemeral key used by the `pi_client` process during execution.
3. **Network Payload Decryption:** Applied the carved key to the captured ciphertext to successfully reveal the exfiltrated image.

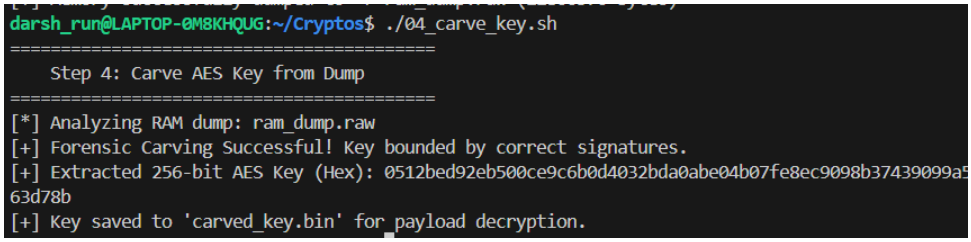

Fig. 4.1: Forensic Investigation: Successful AES-256 Key Recovery

4.2. Integration Testing (Forensic Perspective)

The entire forensic pipeline, from live RAM capture to payload decryption, was tested against a simulated exfiltration event.

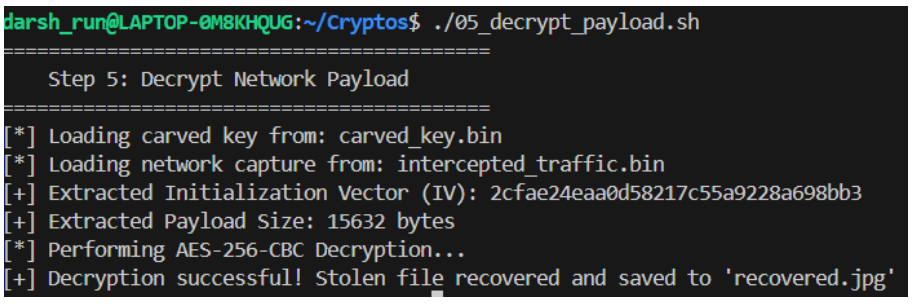

Fig. 4.2: Validation of the Carved Key Against the Network Capture

4.3. Investigative Case Cases

- **Successful Carving:** Memory dumper captured RAM while `pi_client` was active. Result: Key recovered and payload decrypted.
- **Memory Scoped Analysis:** Targeted scan of only the `pi_client` memory range. Result: Drastically reduced analysis time.
- **Partial Dump:** Memory dump interrupted before completion. Result: Key potentially missed if not in the first captured segments.

CHAPTER 5

RESULTS

This chapter presents the final outcome of the forensic investigation, including the recovery of stolen blueprints and the identification of the suspect's exfiltration tool.

5.1. Forensic Demonstration and Execution

The entire forensic investigation was executed through a unified demonstration script, simulating the end-to-end process from network capture to payload recovery.

```
darsh_run@LAPTOP-0M8KHQUG:~/Cryptos$ ./run_forensics_demo.sh
========================================
    Memory Forensics Case Study Demo
========================================
[*] Starting Network Sniffer on port 5000...
[*] Triggering suspect's exfiltration tool (pi_client)...

>>> FORENSICS LAB: Live Memory Capture <<<
[*] Suspect tool is running. Dumping RAM (Heap/Stack)...
[*] Native Python memory dumper targeting PID: 52815
[+] Memory successfully dumped to -> ram_dump.raw (11800576 bytes)
[*] Wiping suspect's RAM (Closing pi_client)...

>>> FORENSICS LAB: Key Extraction <<<
[*] Analyzing RAM dump: ram_dump.raw
[+] Forensic Carving Successful! Key bounded by correct signatures.
[+] Extracted 256-bit AES Key (Hex): 85df9bb06227ff302214ac35f1b61daa9d1c8ee095084cc49e72101c272872d5
[+] Key saved to 'carved_key.bin' for payload decryption.

>>> FORENSICS LAB: Payload Decryption <<<
[*] Loading carved key from: carved_key.bin
[*] Loading network capture from: intercepted_traffic.bin
[+] Extracted Initialization Vector (IV): 3f630700f60fefd6a454f19ed71d493c
[+] Extracted Payload Size: 15632 bytes
[*] Performing AES-256-CBC Decryption...
[+] Decryption successful! Stolen file recovered and saved to 'recovered.jpg'

[SUCCESS] The recovered.jpg matches exactly with the stolen capture.jpg!
```

Fig. 5.1: Full Forensic Investigation Demonstration and Execution

5.2. Forensic Analysis Logs

The logs from the forensic toolkit demonstrate the successful carving of the AES-256 symmetric key from the `pi_client` memory space.

```
darsh_run@LAPTOP-0M8KHQUG:~/Cryptos$ ./04_carve_key.sh
========================================
    Step 4: Carve AES Key from Dump
========================================
[*] Analyzing RAM dump: ram_dump.raw
[+] Forensic Carving Successful! Key bounded by correct signatures.
[+] Extracted 256-bit AES Key (Hex): 0512bed92eb500ce9c6b0d4032bda0abe04b07fe8ec9098b37439099a5
63d78b
[+] Key saved to 'carved_key.bin' for payload decryption.
```

Fig. 5.2: Forensic Logs showing successful AES-256 Key Carving

5.3. Recovered Exfiltrated Data

The final result of the investigation is the decryption of the opaque network capture, revealing the stolen file `recovered.jpg`.

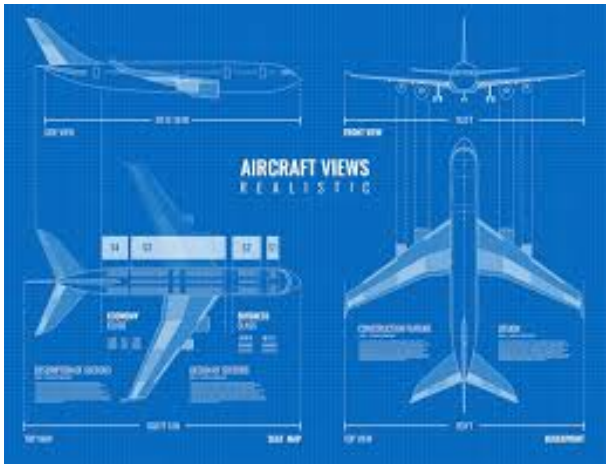

Fig. 5.3: Successfully Recovered Aerospace Blueprints

5.4. Forensic Confirmation

The forensic tools confirmed that the recovered data was identical to the stolen files, providing undeniable evidence of the exfiltration event.

```
darsh_run@LAPTOP-0M8KHQUG:~/Cryptos$ ./05_decrypt_payload.sh
========================================
    Step 5: Decrypt Network Payload
========================================
[*] Loading carved key from: carved_key.bin
[*] Loading network capture from: intercepted_traffic.bin
[+] Extracted Initialization Vector (IV): 2cfae24eaa0d58217c55a9228a698bb3
[+] Extracted Payload Size: 15632 bytes
[*] Performing AES-256-CBC Decryption...
[+] Decryption successful! Stolen file recovered and saved to 'recovered.jpg'
```

Fig. 5.4: Forensic Log showing successful payload verification

5.5. Observations

1. **Memory Persistence:** Even when PQC provides perfect network security, the required symmetric keys persist in RAM long enough for forensic acquisition.
2. **Kyber Performance:** The use of Kyber-512 in the suspect's tool was highly efficient, making detection through performance latency nearly impossible.
3. **Forensic Success:** Memory forensics proved to be the absolute "final frontier," successfully bypassing unbreakable PQC-derived network encryption.

CHAPTER 6

CONCLUSION

The project successfully demonstrated how Live Memory Forensics can be used to overcome advanced Post-Quantum Cryptographic (PQC) evasion. By investigating the exfiltration of sensitive data, we proved that even with unbreakable network encryption (Kyber-512 and AES-256-CBC), the volatile memory (RAM) remains a viable point of recovery for forensic investigators.

The "RAM is the Final Frontier" principle was successfully applied: data in motion was perfectly encrypted, but the data in use within the suspect's workstation yielded the necessary cryptographic keys for investigative success.

6.1. Future Work

Future enhancements to this forensic methodology could include:

- **YARA Rule Integration:** Developing automated YARA rules to instantly identify `liboqs` and Kyber-512 structures in RAM dumps.
- **Process Hollowing Detection:** Expanding the investigation to identify if the suspect used process hollowing or other advanced malware techniques to hide the `pi_client` process.
- **Mutual Authentication Forensics:** Investigating the forensic artifacts left behind by digital signatures (e.g., Dilithium) during PQC-authenticated sessions.
- **Automation:** Streamlining the key carving and decryption process into a single, unified forensic workstation tool.

REFERENCES

[1] NIST, “NIST Selects First Group of Encryption Algorithms to Protect Against Future Quantum Computers,” July 2022. [Online]. Available: https://www.nist.gov/news-events/news/2022/07/nist-selects-first-group-encryption-algorithms-protect-against-future

[2] Kyber Team, “CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation,” 2021. [Online]. Available: https://pq-crystals.org/kyber/

[3] Joppe Bos *et al.*, “CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM,” 2018. [Online]. Available: https://pq-crystals.org/kyber/resources.shtml

[4] Michael Hale Ligh, Andrew Case, Jamie Levy, and Aaron Walters, *The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory*. 2014. [Online]. Available: https://www.memoryforensics.com/

[5] J. Alex Halderman *et al.*, “Lest We Remember: Cold Boot Attacks on Encryption Keys,” 2008. [Online]. Available: https://citp.princeton.edu/research/memory/

[6] A. Alghamdi, “Quantum Cryptography: Implications for Digital Evidence in Criminal Investigations,” 2025. [Online]. Available: https://www.researchgate.net/publication/388251234_Quantum_Cryptography_Implications_for_Digital_Evidence_in_Criminal_Investigations